

Compte Rendu : Projet de Base de Données

"Gestion d'un Restaurant"

INFO0303

2025-2026



Groupe :

AOUANOUK Gaya

Table des matières

Introduction.....	4
Présentation du contexte (Projet de gestion d'un restaurant)	4
Objectifs du projet	4
Outils utilisés (Looping, Wampserver, MySQL).....	4
Annonce du plan	4
1. Étude Préalable et Analyse	5
1.1. Analyse des besoins et des règles de gestion	5
Gestion des Clients et Réservations.....	5
Gestion des Commandes et Paiements.....	5
Gestion de la Carte et des Stocks.....	5
1.2. Matrice des flux entre acteurs	6
2. Modèle Conceptuel de Traitement (MCT).....	7
2.1. Schéma du MCT (Modèle global).....	7
2.2. Description des processus principaux	8
3. Modèle Conceptuel de Données (MCD)	8
3.1. Schéma du MCD (issu de Looping)	8
3.2. Description des entités et associations (justification des choix)	9
3.3. Dictionnaire de données (principales entités)	10
4. Modèle Logique de Données (MLD) / Relationnel	11
4.1. Règles de passage du MCD au MLD.....	11
4.2. Schéma relationnel (la liste de vos tables avec clés primaires et étrangères).....	11
4.3. Analyse de la normalisation (validation 3NF).....	12
5. Requêtes Algèbre Relationnelle	13
5. Interrogation de la Base de Données (Requêtes SQL)	16
5.1. Requête imbriquée	16
Requête de calcul	16
Requête planning	17
Requête de soustraction relationnelle	17
Requête de division relationnelle.....	17
Conclusion	19

Introduction

Présentation du contexte (Projet de gestion d'un restaurant)

Ce document présente la conception et la réalisation d'une base de données destinée à la gestion complète d'un restaurant. Le besoin initial est de centraliser et de fiabiliser l'ensemble des opérations quotidiennes, allant de la prise de réservation à l'encaissement, en passant par la gestion des stocks et des livraisons.

Face à la complexité des flux d'information à gérer simultanément (clients, commandes en salle, livraisons, personnel, stocks d'ingrédients, fournisseurs), la mise en place d'un Système de Gestion de Base de Données (SGBD) relationnel est apparue comme la solution indispensable pour structurer l'information et optimiser le fonctionnement de l'établissement.

Objectifs du projet

L'objectif principal est de mettre en œuvre une base de données relationnelle fonctionnelle. Pour y parvenir, nous avons suivi la méthodologie MERISE, en respectant les étapes suivantes :

- Analyse des besoins et modélisation des flux d'information (MCT).
- Conception d'un Modèle Conceptuel de Données (MCD) stable et normalisé.
- Traduction du MCD en Modèle Logique de Données (MLD).
- Implémentation physique de la base sous MySQL (LDD).
- Insertion d'un jeu d'essai (LMD) pour valider le modèle.
- Interrogation de la base à l'aide de requêtes SQL complexes.

Outils utilisés (Looping, Wampserver, MySQL)

Pour la réalisation de ce projet, les outils suivants ont été utilisés, conformément aux directives [cite: 1445-1447] :

- **Looping** : Logiciel de modélisation MERISE utilisé pour la création des schémas MCD et MCT.
- **Wampserver** : Environnement de développement local incluant Apache, PHP, et MySQL.
- **MySQL** : Système de Gestion de Base de Données (SGBD) relationnel pour l'implémentation physique de notre base.
- **phpMyAdmin** : Interface web pour l'administration de la base de données et l'exécution des requêtes SQL.

Annonce du plan

Ce compte rendu suivra les étapes logiques de la conception de la base de données. Nous commencerons par l'étude préalable et l'analyse des besoins, suivies de la présentation de nos modèles (MCT et MCD). Nous détaillerons ensuite le passage au Modèle Logique (MLD) et son implémentation (LDD, LMD). Enfin, nous démontrerons l'efficacité de notre base en présentant les 5 requêtes SQL spécifiées dans le cahier des charges.

1. Étude Préalable et Analyse

1.1. Analyse des besoins et des règles de gestion

Avant de modéliser, une analyse des besoins du restaurant a permis de définir un ensemble de règles de gestion (RG) qui dictent le fonctionnement du système. Ces règles sont directement issues des processus métier de l'établissement et constituent le fondement du Modèle Conceptuel de Données (MCD). Les règles de gestion principales identifiées sont les suivantes :

Gestion des Clients et Réservations :

- Un client peut effectuer plusieurs réservations (0,n), mais peut aussi être un client de passage sans réservation.
- Une réservation est effectuée par un seul client (0,1). Elle peut être enregistrée sans être rattachée à un compte client (ex: réservation téléphonique).
- Une réservation doit concerner une et une seule table pour être valide (1,1).
- Une table peut être l'objet de plusieurs réservations (0,n) à des dates ou heures différentes.

Gestion des Commandes et Paiements :

- Une commande est prise en charge par un seul serveur (0,1). Une commande peut n'avoir aucun serveur (ex: commande en ligne). Un serveur peut gérer plusieurs commandes (0,n).
- Une commande génère une et une seule addition (1,1). Une addition correspond à une et une seule commande (1,1). *(Note : Le MCD (0,1) suggère qu'une commande peut exister sans addition, mais le MR et le SQL (FK UNIQUE) implémentent une relation 1,1).*
- Une commande peut concerner une livraison (0,1) ou être consommée sur place (pas de livraison).
- Une livraison est associée à une seule commande (0,1). *(Note : la cardinalité (0,1) suggère qu'une livraison peut être planifiée avant même que la commande ne soit finalisée).*
- Une commande peut contenir plusieurs plats (0,n) et/ou plusieurs menus (0,n).
- Une commande doit contenir au minimum soit un plat, soit un menu.

Gestion de la Carte et des Stocks :

- Un menu est composé de plusieurs plats (1,n). Un plat peut être inclus dans plusieurs menus (0,n).
- Un plat est composé d'un ou plusieurs ingrédients (1,n).
- Un ingrédient peut être utilisé dans la composition de plusieurs plats (0,n).
- Un ingrédient peut être fourni par plusieurs fournisseurs (1,n).
- Un fournisseur propose un ou plusieurs ingrédients (1,n) à son catalogue.

1.2. Matrice des flux entre acteurs

Nous avons identifié quatre acteurs principaux interagissant avec le système. La matrice des flux ci-dessous résume les principaux échanges d'informations.

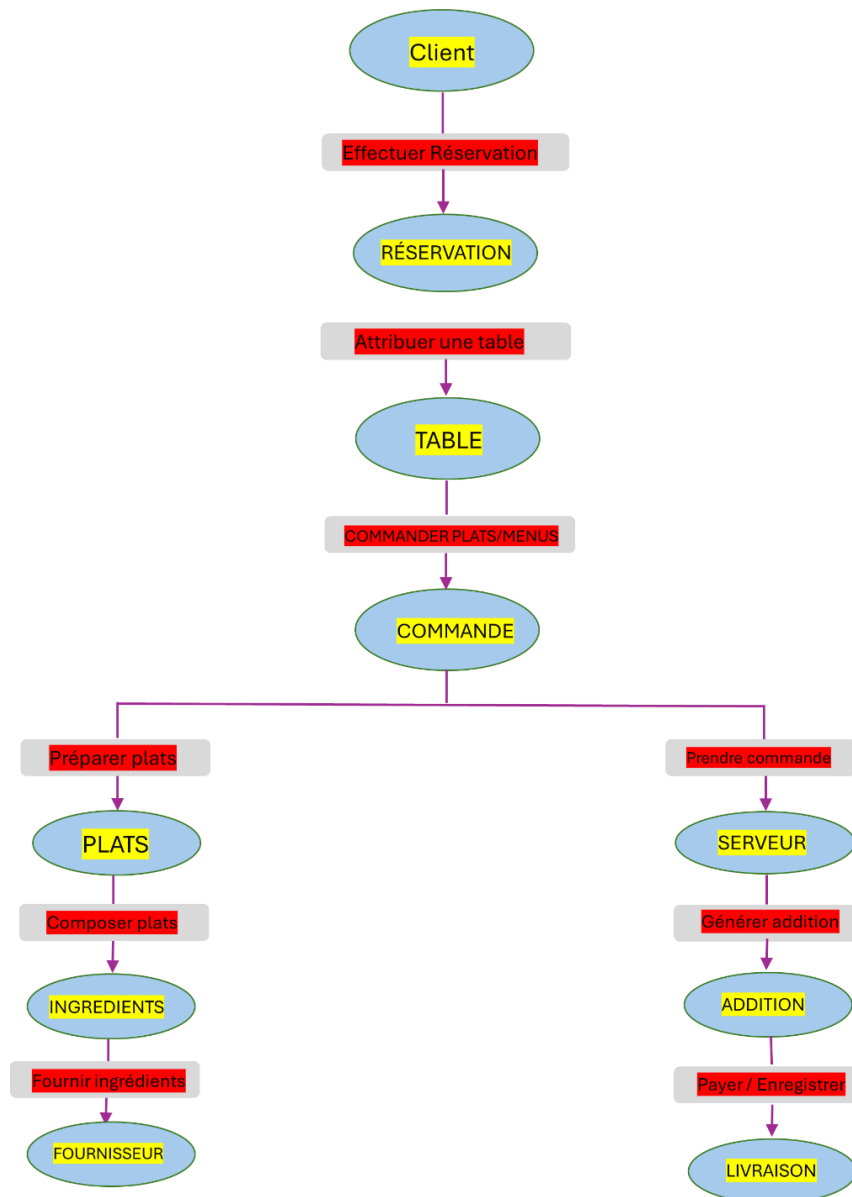
Usager	Client	Serveur	Cuisine	Système d'informations
Client		Demande une réservation Passe une commande		envoi des informations de réservation (nom, nombre de personnes)
Serveur	Confirme la réservation Remet l'addition		envoi de la commande à préparer	Enregistre commande Associe plats/menus Génère addition
Cuisine		notification que les plats sont prêts		signalement de manque d'ingrédients mise à jour de l'état de préparation
Système d'informations	Envoie notification de confirmation	Validation de commande Addition générée		

2. Modèle Conceptuel de Traitement (MCT)

Le MCT décrit la dynamique du système, c'est-à-dire les processus métiers et leur enchaînement, en réponse à des événements.

2.1. Schéma du MCT (Modèle global)

Le MCT a été modélisé pour représenter les processus majeurs : "Gérer les réservations", "Traiter une commande" (qui se divise en "Commande sur place" et "Commande à livrer"), et "Gérer les paiements".



2.2. Description des processus principaux

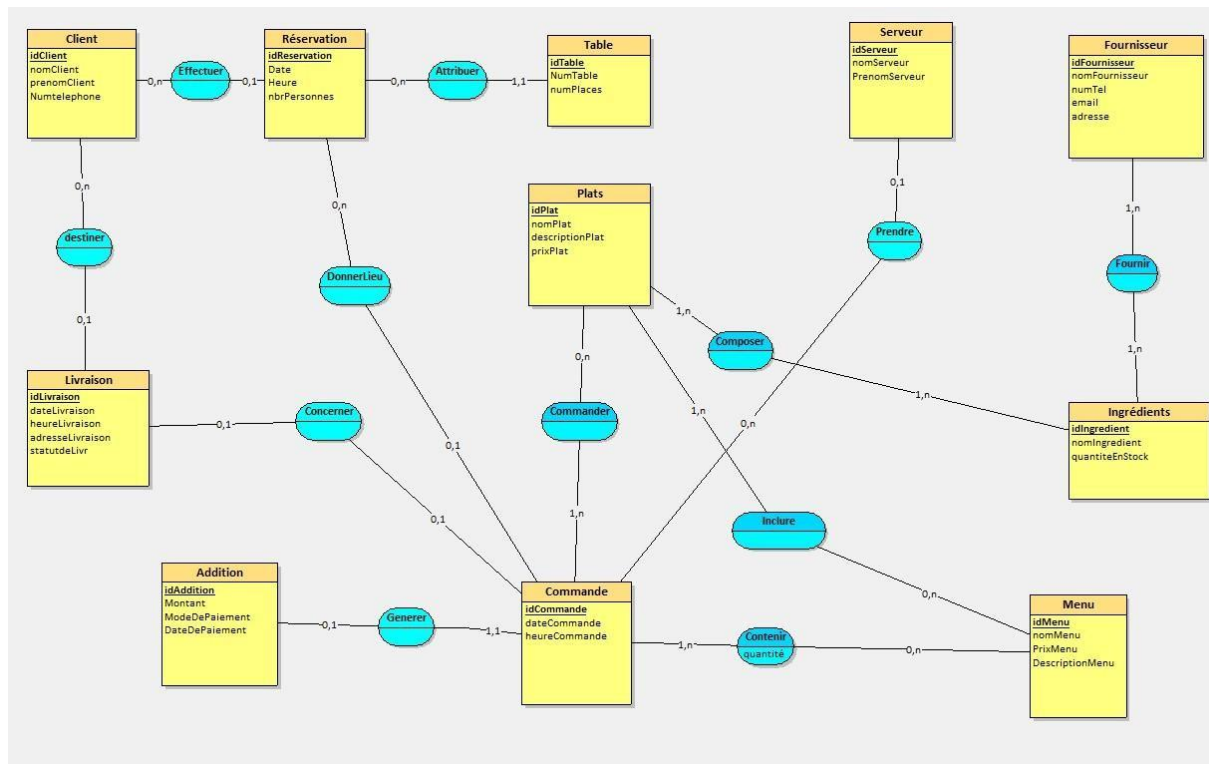
- **Processus "Gérer une réservation" :**
 - **Événement :** "Demande de Réservation" du Client.
 - **Opérations :** Vérifier disponibilité table, enregistrer réservation, assigner table.
 - **Résultat :** "Confirmation Réservation".
- **Processus "Traiter une commande" :**
 - **Événement :** "Prise de Commande" par le Serveur ou "Demande de Livraison" par le Client.
 - **Opérations :** Créer commande, lier plats/menus, créer addition (statut "non payée").
 - **Résultat :** "Commande enregistrée", "Addition créée".
- **Processus "Gérer le paiement" :**
 - **Événement :** "Demande de paiement" du Client.
 - **Opérations :** Mettre à jour l'addition (Montant, ModeDePaiement, DateDePaiement).
 - **Résultat :** "Paiement confirmé".

3. Modèle Conceptuel de Données (MCD)

Le MCD est une représentation statique des données et de leurs relations, basée sur les règles de gestion identifiées.

3.1. Schéma du MCD (issu de Looping)

Notre MCD est centré sur l'entité Commande, qui fait le lien entre les Clients (via Réservation ou Livraison), les Serveurs, les Additions, et ce qui est consommé (Plats, Menu) :



3.2. Description des entités et associations (justification des choix)

Basé sur le LDD fourni, le modèle se compose de :

- **Entités (Entités fortes) :**
 - Client : Stocke les informations des clients.
 - Serveur : Répertorie les employés.
 - Fournisseur : Répertorie les fournisseurs d'ingrédients.
 - Plats : Définit les plats à la carte (prix, description).
 - Menu : Définit les formules (prix, description).
 - Ingrédients : Gère les ingrédients en stock.
 - Addition : Contient les informations de paiement (montant, date).
- **Entités (Entités faibles / Associations porteuses de données) :**
 - Réserve : Liée à un Client (1,N).
 - Table_ : Liée à une Réserve (1,N).
 - Commande : Liée à une Addition (1,1, UNIQUE), et optionnellement à une Réserve (0,N).
 - Livraison : Liée à un Client et une Commande.
- **Associations (N,N) devenant des tables :**

- Contenir (Commande, Menu) : Permet de commander plusieurs fois le même menu (quantité).
- Commander (Commande, Plats) : Lie les plats à une commande.
- Inclure (Menu, Plats) : Définit la composition d'un menu.
- Composer (Plats, Ingrédients) : Définit la recette d'un plat.
- Fournir (Fournisseur, Ingrédients) : Gère l'approvisionnement.

3.3. Dictionnaire de données (principales entités)

Entité	Attribut	Type de données	Clé (PK/FK)	Description
Client	idClient	BIGINT	PK	Identifiant unique du client
	nomClient	VARCHAR(50)		Nom de famille
	prenomClient	VARCHAR(50)		Prénom
	Numtelephone	VARCHAR(20)		Numéro de téléphone
Plats	idPlat	BIGINT	PK	Identifiant unique du plat
	nomPlat	VARCHAR(80)		Nom du plat
	descriptionPlat	VARCHAR(150)		Description
	prixPlat	DOUBLE		Prix unitaire
Commande	idCommande	INT	PK	Identifiant unique de la commande
	dateCommande	DATE		Date de la prise de commande

	heureCommande	TIME		Heure de la prise de commande
	idAddition	INT	FK (Unique)	Lien vers l'addition (paiement)
	idReservation	BIGINT	FK	Lien vers la réservation (si sur place)

4. Modèle Logique de Données (MLD) / Relationnel

Le MLD est la traduction du MCD en un modèle relationnel, préparant l'implémentation physique.

4.1. Règles de passage du MCD au MLD

Nous avons appliqué les règles de passage standards :

1. Chaque entité du MCD devient une table dans le MLD (Client, Plats...).
2. Chaque association de type (N,N) ou (N,M) devient une table (Inclure, Composer, Contenir, Commander, Fournir). La clé primaire de ces tables est la concaténation des clés primaires des entités qu'elles relient.
3. Pour les associations de type (1,N), la clé primaire de l'entité côté (1) migre en tant que clé étrangère (FK) dans la table de l'entité côté (N). (Ex: idClient de Client (1) migre dans Réservation (N)).
4. L'association (1,1) entre Commande et Addition est gérée par une clé étrangère idAddition dans Commande, avec une contrainte UNIQUE pour garantir la cardinalité (1,1).

4.2. Schéma relationnel (la liste de vos tables avec clés primaires et étrangères)

Voici le Modèle Relationnel (MR) textuel, où PK désigne une Clé Primaire et FK une Clé Étrangère.

Client = (idClient BIGINT, nomClient VARCHAR(50), prenomClient VARCHAR(50), Numtelephone VARCHAR(20));

Addition = (idAddition INT, Montant DECIMAL(10,2), ModeDePaiement VARCHAR(25), DateDePaiement DATE);

Réservation = (idReservation BIGINT, Date_ DATE, Heure TIME, nbrPersonnes BIGINT, #idClient*);

Menu = (idMenu BIGINT, nomMenu VARCHAR(50), PrixMenu DOUBLE, DescriptionMenu VARCHAR(150));

Table_ = (idTable BIGINT, NumTable INT, numPlaces INT, #idReservation);

Fournisseur = (idFournisseur INT, nomFournisseur VARCHAR(50), numTel VARCHAR(20), email VARCHAR(50), adresse VARCHAR(100));

Plats = (idPlat BIGINT, nomPlat VARCHAR(80), descriptionPlat VARCHAR(150), prixPlat DOUBLE);

Ingrédients = (idIngredient VARCHAR(50), nomIngredient VARCHAR(50), quantiteEnStock VARCHAR(50));

Commande = (idCommande INT, dateCommande DATE, heureCommande TIME, #idAddition, #idReservation*);

Livraison = (idLivraison VARCHAR(50), dateLivraison DATE, heureLivraison TIME, adresseLivraison VARCHAR(100), statutdeLivraison VARCHAR(50), #idClient*, #idCommande*);

Serveur = (idServeur BIGINT, nomServeur VARCHAR(50), PrenomServeur VARCHAR(50), #idCommande*);

Contenir = (#idCommande, #idMenu, quantité BIGINT);

Commander = (#idCommande, #idPlat);

Inclure = (#idMenu, #idPlat);

Composer = (#idPlat, #idIngredient);

Fournir = (#idFournisseur, #idIngredient);

4.3. Analyse de la normalisation (validation 3NF)

Notre schéma relationnel respecte la Troisième Forme Normale (3NF).

- **Première Forme Normale (1NF) :** Toutes les tables ont une clé primaire et tous les attributs sont atomiques (non décomposables).
- **Deuxième Forme Normale (2NF) :** Le modèle respecte la 2NF. Pour les tables à clé primaire simple (ex: Plats), la 2NF est trivialement respectée. Pour les tables à clé composite (ex: Contenir), l'attribut quantité dépend bien de la totalité de la clé (idCommande ET idMenu), et non d'une partie seulement.
- **Troisième Forme Normale (3NF) :** Le modèle respecte la 3NF. Il n'existe pas de dépendance transitive. Par exemple, dans la table Plats, nomPlat et prixPlat dépendent

directement de idPlat. Il n'y a pas d'attribut non-clé qui dépendrait d'un autre attribut non-clé.

5. Requêtes Algèbre Relationnelle :

Chaque requête est exprimée à l'aide des opérations fondamentales : sélection (σ), projection (π) et jointure ($\bowtie$) :

Liste des clients ayant fait une réservation :

Algèbre relationnelle :

```
 $\pi_{\text{nomClient}, \text{prenomClient}} ( \text{Client} \bowtie_{\{\text{Client.idClient} = \text{Réservation.idClient}\}} \text{Réservation} )$ 
```

Commandes passées par un client donné

Algèbre relationnelle :

```
 $\pi_{\text{idCommande}, \text{dateCommande}, \text{nomClient}} ($   
   $\sigma_{\text{nomClient} = \text{'MOUNIR'}} ($   
     $(\text{Commande} \bowtie_{\{\text{Commande.idReservation} = \text{Réservation.idReservation}\}} \text{Réservation})$   
     $\bowtie_{\{\text{Réservation.idClient} = \text{Client.idClient}\}} \text{Client}$   
  )  
)
```

Les plats composant un menu donné

Algèbre relationnelle :

```
 $\pi_{\text{nomMenu}, \text{nomPlat}} ($   
   $\sigma_{\text{nomMenu} = \text{'Menu TRADITIONNEL'}} ($   
     $(\text{Menu} \bowtie_{\{\text{Menu.idMenu} = \text{Inclure.idMenu}\}} \text{Inclure})$   
     $\bowtie_{\{\text{Inclure.idPlat} = \text{Plats.idPlat}\}} \text{Plats}$   
  )
```

))

Fournisseurs livrant un ingrédient donné

Algèbre relationnelle :

```
π_nomFournisseur, nomIngrédient (  
  σ_nomIngrédient='Tomate' (  
    (Fournisseur ⋈_{Fournisseur.idFournisseur = Fournir.idFournisseur} Fournir)  
    ⋈_{Fournir.idIngrédient = Ingrédients.idIngrédient} Ingrédients  
  )  
)
```

Ingrédients utilisés dans un plat

Algèbre relationnelle :

```
π_nomFournisseur, nomIngrédient (  
  σ_nomIngrédient='Tomate' (  
    (Fournisseur ⋈_{Fournisseur.idFournisseur = Fournir.idFournisseur} Fournir)  
    ⋈_{Fournir.idIngrédient = Ingrédients.idIngrédient} Ingrédients  
  )  
)
```

Fournisseurs livrant un ingrédient donné

Algèbre relationnelle :

```
π_nomFournisseur, nomIngrédient (  
  σ_nomIngrédient='Tomate' (  
    (Fournisseur ⋈_{Fournisseur.idFournisseur = Fournir.idFournisseur} Fournir)  
    ⋈_{Fournir.idIngrédient = Ingrédients.idIngrédient} Ingrédients  
  )  
)
```

Ingrédients utilisés dans un plat

Algèbre relationnelle :

```
π_nomFournisseur, nomIngrédient (  
  σ_nomIngrédient='Tomate' (  
    (Fournisseur ⋈_{Fournisseur.idFournisseur = Fournir.idFournisseur} Fournir)  
    ⋈_{Fournir.idIngrédient = Ingrédients.idIngrédient} Ingrédients  
  )  
)
```

Serveurs ayant pris des commandes

Algèbre relationnelle :

```
 $\pi_{\text{nomServeur, PrenomServeur, idCommande}} ($   
   $\text{Serveur} \bowtie_{\{\text{Serveur.idServeur} = \text{Commande.idServeur}\}} \text{Commande}$   
 $)$ 
```

Les menus commandés dans une commande donnée

Algèbre relationnelle :

```
 $\pi_{\text{nomMenu}} ($   
   $\sigma_{\text{idCommande}=5} ( \text{Menu} \bowtie_{\{\text{Menu.idMenu} = \text{Contenir.idMenu}\}} \text{Contenir} )$   
 $)$ 
```

5. Interrogation de la Base de Données (Requêtes SQL)

5.1. Requête imbriquée

Objectif : Afficher les noms et prénoms des clients qui ont effectué une réservation pour 4 personnes ou plus.

Code SQL :

```
SELECT nomClient, prenomClient, Numtelephone  
      FROM Client  
      WHERE idClient IN (  
        SELECT idClient  
      FROM Réservation  
        WHERE nbrPersonnes >= 4  
      );
```

Résultat attendu : Affiche le Client 2 'SERRAYE MOUNIR' basé sur le jeu d'essai

Requête de calcul

Objectif : Calculer le montant total encaissé (via les additions) pour chaque mode de paiement.

Code SQL :

```
SELECT ModeDePaiement, SUM(Montant) AS MontantTotal
```



```
FROM Addition  
GROUP BY ModeDePaiement;
```

(Résultat attendu : Affiche 'CB' -> 39.00 et 'Espèces' -> 12.00)

Requête planning

Objectif : Afficher le planning des réservations pour la journée du '2025-11-11' (nom du client, heure, nombre de personnes et numéro de table).

Code SQL :

```
SELECT C.nomClient, R.Date_, R.Heure, R.nbrPersonnes, T.NumTable
```

```
FROM Réservation R
```

```
JOIN Client C ON R.idClient = C.idClient
```

```
JOIN Table_ T ON R.idReservation = T.idReservation
```

```
WHERE R.Date_ = '2025-11-11'
```

```
ORDER BY R.Heure;
```

Résultat attendu : Affiche la réservation de 'Mounir' à 12:30 pour 5 personnes à la table 10)

Requête de soustraction relationnelle

Objectif : Trouver les plats (nom et prix) qui ne sont inclus dans *aucun* menu.

Code SQL :

```
SELECT nomPlat, prixPlat
```

```
FROM Plats
```

```
WHERE idPlat NOT IN (  
SELECT DISTINCT idPlat
```

```
FROM Inclure  
);
```

Résultat attendu : Affiche 'Petit Coca' et 'Petit Fanta' si le jeu d'essai est modifié pour les exclure, ou rien si tous les plats sont dans un menu. Avec le LMD actuel, cette requête ne retourne rien car tous les plats sont dans au moins un menu.

Requête de division relationnelle

Objectif : Trouver les menus (id et nom) qui incluent *tous* les plats "boisson" (ceux dont le prix est inférieur à 4.00 €).

Code SQL :

```
SELECT m.idMenu, m.nomMenu
```

```
FROM Menu m
```

```
WHERE NOT EXISTS (
```

1. *L'ensemble des plats "diviseurs" (boissons < 4.00€)*

```
SELECT p.idPlat
```

```
FROM Plats p
```

```
WHERE p.prixPlat < 4.00
```

```
EXCEPT
```

2. *L'ensemble des plats que ce menu 'm' inclut déjà*

```
SELECT i.idPlat
```

```
FROM Inclure i
```

```
WHERE i.idMenu = m.idMenu
```

```
);
```

Résultat attendu : Affiche 'Menu Enfant' (ID 20), car il contient le Fanta (3.50) et le Coca (3.50), qui sont les deux seuls plats à moins de 4.00€ dans le jeu d'essai. Le 'Menu du Jour' (ID 10) ne

les contient pas

Conclusion

Bilan du travail réalisé

Ce projet nous a permis de mettre en œuvre l'intégralité du processus de conception de bases de données, depuis l'analyse des besoins jusqu'à l'implémentation physique et les requêtes complexes. Il en résulte un système d'information bien structuré, conforme aux normes de normalisation (3NF), qui répond efficacement aux besoins opérationnels d'un restaurant. Les scripts DDL et DML fournis permettent la création et l'alimentation d'une base de données fonctionnelle.

Difficultés rencontrées

Les principales difficultés ont été :

1. **Modélisation (MCD)** : La distinction correcte entre les commandes sur place (liées à une Réservation) et les Livraisons, tout en les reliant à une Commande unique, a nécessité une attention particulière aux cardinalités (notamment les (0,1)).
2. **Requêtes SQL** : La conception de la requête de division relationnelle (6.5) est complexe. Elle exige une traduction correcte de l'expression "pour tous" en une double négation (NOT EXISTS ... EXCEPT ...) en SQL.

Pistes d'amélioration futures

Le modèle actuel est robuste, mais pourrait être étendu pour inclure de nouvelles fonctionnalités :

- **Gestion des stocks** : Mettre en place des triggers pour décrémenter automatiquement `quantiteEnStock` de la table `Ingrédients` lors de l'insertion dans `Commander`.
- **Gestion du personnel** : Étendre l'entité `Serveur` pour y inclure la gestion des plannings, des rôles (cuisinier, manager), et une meilleure assignation aux tables (`Table_`) plutôt qu'aux commandes.
- **Système de notation** : Permettre aux Clients de noter les Plats ou leur expérience (liée à l' `Addition`)

